Group 20 Work Distribution

Change-Aware, Feasibility-First Scheduling for AI Inference across Geo-Distributed Nodes

AMS 560 • Big Data Systems, Algorithms and Networks • Fall 2026
Based on: the Sept 17 proposal, the 8-slide proposal deck, and the professor's two comments on it.

1. What we are building, in one paragraph

A company runs the same AI model at 5 to 10 places (nodes). Every request has to be sent to one of them. Our scheduler does this in four steps: **filter** out nodes that cannot or must not take the request (model missing, no memory, no power headroom, too hot); **choose** among the rest with sliding-window LinUCB, using request and node information; **send** the request; **learn** from the outcome, keeping only recent results so stale history is forgotten. The scheduler is real code (an HTTP service). The nodes are simulated. We test it under six controlled changes and measure how fast it recovers.

2. The professor's comments and who fixes them

Comment	What it means for us	Owner
Frame the novelty around the full scheduling system and the recovery evaluation, not around sliding-window LinUCB itself.	Sliding-window LinUCB is an existing algorithm (we cite it). Our claim is the integrated loop (filter + context + forgetting) and the recovery benchmark. Reword slide 4, contribution 2, and the report's contribution list.	Krupesh (wording) Sur (citations)
Define "adaptation time" precisely, e.g. time to return within a fixed percentage of the new steady-state goodput after a change.	Write one exact definition, freeze it, and use it for every run. Proposed definition is in Section 6.	Riya (metric code) All (sign-off)

3. Who owns what (summary)

Person	Primary area	Main output
Krupesh	Architecture, router service, integration	API contract + working end-to-end system
Parikshit	Node simulator + telemetry + event traces	Simulator that produces node numbers over time
Jyotir	Feasibility filter + static baselines	Filter module + 5 baseline policies
Sur	LinUCB + sliding-window policy + reward	Policy module (plain and windowed)
Riya	Experiments, metrics, analysis, report	Result bundle, figures, written findings
All	Validation, final review, recording	Submission-ready package

The Sept 17 proposal had three roles (A: math and algorithm, B: node simulation, C: benchmarking). They map to Sur, Parikshit and Riya. Krupesh and Jyotir take the two pieces added in the slides: integration and the feasibility filter with baselines.

4. Detailed tasks per person

Krupesh — Architecture + integration

Owns: the overall design, the HTTP router, the interfaces between modules, and the integrated build.

Tasks

- Write the **routing API contract**: what a request looks like (model, input length, output length, class, SLO) and what the router returns (chosen node or rejection + reason).
- Build the **router service** (e.g. FastAPI). It receives a request, reads telemetry, calls the filter, calls the policy, logs the decision.
- Define the **decision log** format: one row per request with time, chosen node, filtered-out nodes and reasons, latency, SLO met, cost.
- Make policies **pluggable**: any policy (baseline or LinUCB) has the same interface, so experiments can swap them with one config line.
- Run the **weekly integration check**: pull everyone's module, run one end-to-end scenario, report what breaks.
- Rewrite the **novelty wording** on slide 4 and in the report per the professor's first comment.

Deliverables	Needs from	Hands to
• API contract (written doc) • Router service code • Decision log format • Integrated build each week	Telemetry schema (Parikshit), filter (Jyotir), policy (Sur)	Running system + logs to Riya

Report / presentation part: Introduction, problem, system design section; design slide.

Parikshit — Simulator + telemetry

Owns: the simulated nodes and everything they report over time.

Tasks

- Build a simulator of **5 to 10 nodes** (e.g. with SimPy). Each node has: network delay to users, queue, number of running requests, GPU memory used, models loaded, power headroom, price, temperature.
- Write a simple **latency model per node**: time = network delay + queue wait + prefill time (grows with input length) + decode time (grows with output length, slower when the node is busier). Use the book, sections 2.2, 2.4 and 7.2.1.
- Generate requests from **real traces** where possible (Azure LLM inference traces for arrival times and token lengths).
- Use real public data where it exists: hourly electricity prices (NYISO / CAISO), city-to-city network delay (public ping data).
- Script the **six change events** as replayable traces: stable, traffic burst, price/renewable shift, thermal throttle, network degradation, outage + recovery (a recovered node is not instantly ready: add warm-up).
- Fix a **random seed** per run so every result can be reproduced exactly.
- Write the **telemetry schema** (versioned) and unit tests for the simulator.

Deliverables	Needs from	Hands to
• Simulator code + tests • Telemetry schema v1 • 6 event trace files • Short note: how each number is produced	API contract (Krupesh)	Telemetry to router, filter and policy; traces to Riya

Report / presentation part: Simulator and workload section; explains why the simulated numbers are believable.

Jyotir — Feasibility filter + static baselines

Owns: the hard safety rules and all the simple policies we compare against.

Tasks

- Write the **feasibility rules**: a node is allowed only if it has the model, enough GPU memory, power headroom above a limit, and temperature below a limit. Put limits in a config file, not in code.
- Return a **reason** for every node removed, and a controlled **rejection** when no node is left.
- Build the **static baselines**: round-robin, minimum RTT (closest), least load, lowest energy cost, and a fixed weighted rule.
- Make sure baselines also pass through the **same filter**, so comparisons are fair (only the choosing step differs).
- Optional, cheap extra baseline: **power of two choices** (pick 2 random feasible nodes, send to the less busy one).
- Run all baselines on the **same event traces** as the adaptive policy and keep the logs (the baseline log).
- Test edge cases: all nodes too hot, only one node feasible, node recovering from outage.

Deliverables	Needs from	Hands to
• Feasibility module + tests • 5 (or 6) baseline policies • Baseline log on all 6 traces • Rule/threshold table for the report	Telemetry schema (Parikshit), policy interface (Krupesh)	Filter to Sur and router; baseline log to Riya

Report / presentation part: Feasibility and baselines section; slide on the filter and the rejection path.

Sur — LinUCB + sliding-window policy

Owns: the learning policy and the reward it learns from.

Tasks

- Implement **plain LinUCB** (Li et al., 2010): one linear model per node, choose highest score + exploration bonus.
- Implement **sliding-window LinUCB**: keep only the last W outcomes per node (Cheung, Simchi-Levi, Zhu, 2019). Plain LinUCB stays as an ablation.
- Choose only **among nodes the filter allows**.
- Write the **context vector** (request size, class, SLO, node delay, queue, memory, price, temperature), with scaling so features are comparable.
- Write the **reward**: SLO met − latency − energy cost − thermal stress, with weights in config.
- Run a **window-size sweep** (small W reacts fast but is noisy, large W is stable but slow) and an exploration-parameter check.
- Provide **citations** for LinUCB and the sliding-window idea so the report credits them correctly.

Deliverables	Needs from	Hands to
• Policy module (plain + windowed) • Reward function • Parameter file + checkpoints • Window-size sweep results	Telemetry (Parikshit), filter (Jyotir), interface (Krupesh)	Policy to router; sweep results to Riya

Report / presentation part: Method section (policy, context, reward); related-work paragraph on bandits.

Riya

Experiments + analysis + report

Owns: running all experiments, the metrics, the figures and the final report.

Tasks

- Implement the **metrics** from the decision log: SLO goodput, p95 / p99 latency, rejection rate, cost and energy per successful request, and **adaptation time** (Section 6).
- Build the **experiment runner**: every policy × every scenario × at least 10 seeds, from one config file.
- Compute **95% bootstrap confidence intervals**.
- Run the **ablations**: no filter, no context (plain bandit), no forgetting (plain LinUCB).
- Make the **figures**: goodput over time around each change event (with the adaptation-time marker), and bar charts with confidence intervals.
- Assemble the **report** from everyone's sections; write results and discussion, including where the method does not help.

Deliverables	Needs from	Hands to
• Metrics code incl. adaptation time • Experiment runner + configs • Result bundle (raw logs + tables) • Figures + report draft	Logs from router, traces, baseline log, sweep results	Final report and figures to all

Report / presentation part: Evaluation, results, discussion; results slides.

5. Shared contracts (agree these first)

Each person can work alone as long as these four contracts stay fixed. Any change to one needs a message to the whole group.

Contract	Owner	Used by	Due
Telemetry schema (what every node reports)	Parikshit	Krupesh, Jyotir, Sur	End of Week 1
Routing API + policy interface	Krupesh	Everyone	End of Week 1
Decision log format	Krupesh	Riya	End of Week 1
Experiment bundle (configs, seeds, raw logs, tables)	Riya	Everyone (report)	End of Week 3

6. Definitions to freeze

Goodput: requests per second that finish within their SLO (TTFT under 250 ms in the slides). Late requests do not count.

Adaptation time (answers the professor's second comment). Proposed, for group sign-off:

> A change happens at time t_0. Measure goodput G(t) in 1-second bins, smoothed over 5 seconds. Let G* be the average goodput from 60 s to 120 s after t_0 (the new steady state). **Adaptation time** = the first moment after t_0 when G(t) reaches at least 90% of G* **and stays there for 10 seconds**, minus t_0. A run that never gets there is reported as "did not recover within the run", not as a number.

Also report, for every change event: the **deepest dip** in goodput, and the **total lost goodput** (area between the best-possible curve and ours during recovery). Because a weak policy can settle at a low G* and look fast, also show each policy's G* next to its adaptation time. The 90%, 10 s and window values are starting choices: pick once, then keep them fixed for all runs.

Reward: SLO met − latency − energy cost − thermal stress (weights fixed in config).

Novelty statement for the report: "We contribute an integrated, feasibility-first, change-aware scheduling loop for multi-site inference and a recovery benchmark with six controlled change events. Sliding-window LinUCB is an existing method that we use inside this loop."

7. Timeline (from slide 8)

The slides plan four weeks with a core freeze at the end of Week 3. Week dates are not fixed here; set them in the first meeting.

	Week 1: Foundation	Week 2: Baseline system	Week 3: Adaptive system	Week 4: Buffer + finalize
Krupesh	API contract, log format, router skeleton	Router + filter + baselines wired end to end	LinUCB plugged in; integration fixes	Final review; design section
Parikshit	Telemetry schema; 5 nodes running; tests	Latency model; real traces loaded	6 event traces finished	Fix gaps; simulator section
Jyotir	Feasibility rules + config	5 baselines running through the filter	Baselines on all 6 traces; edge tests	Re-runs; baselines section
Sur	Context vector + reward draft	Plain LinUCB working	Sliding window; W sweep	Re-runs; method section
Riya	Metrics spec; adaptation-time code	Runner + first baseline numbers	Full runs, CIs, ablations	Figures, report, results slides
All	Freeze the 4 contracts	End-to-end demo check	**Core freeze**	Zoom recording + submission

8. Done means

- **Better:** higher SLO goodput or lower cost than the strongest baseline, with confidence intervals.
- **Faster:** shorter adaptation time than plain LinUCB after change events.
- **Safe:** no hard constraint is ever broken to save cost.
- **Reproducible:** anyone can rerun every figure from the configs and seeds.
- A negative result is acceptable if the experiment is clean and explains when the method helps and when it does not.